

INTERNET OF THINGS (IoT)

Unit II — Comprehensive Study Notes

Sensors, Actuators, and Microcontrollers

TY BSC IT | Exam Preparation Guide

Cognitive Level: K1, K2, K3, K4 | Hours: 12 | PO: 1, 2, 3, 4, 5, 7, 11

- ✓ Types of Sensors — Temperature, Motion, Proximity, Gas
 - ✓ Actuators — Motors, Relays, Valves
- ✓ Introduction to Arduino and Raspberry Pi
 - ✓ GPIO Programming
 - ✓ Sensor-Controller Interfacing

1. Sensors in IoT

Sensor — Formal Definition

A sensor is a device that detects or measures a physical property (such as temperature, light, pressure, motion, or gas concentration) from the environment and converts it into an electrical signal (analog voltage or digital output) that can be read by a microcontroller or computer for processing.

Analogy: Sensors are the 'sense organs' of an IoT system. Just as human senses (sight, hearing, touch, smell, taste) gather information about the world, sensors gather physical data for the IoT system to act upon.

1.1 Sensor Classification

Classification Basis	Types	Examples
Output Signal Type	Analog (continuous voltage output) vs Digital (0 or 1 output)	LM35 = analog; DHT11 = digital
Energy Source	Active (needs external power) vs Passive (self-powered by measured phenomenon)	Thermistor = passive; Ultrasonic = active
Physical Quantity Measured	Temperature, Light, Pressure, Motion, Proximity, Gas, Humidity, Sound	See all categories below
Contact with Measurand	Contact (physically touches what it measures) vs Non-contact	Thermocouple = contact; IR sensor = non-contact

1.2 Temperature Sensors

Purpose: Measure the thermal energy (heat) of an object or environment and convert it into an electrical signal.

LM35 — Analog Temperature Sensor

- Type: Analog output, integrated circuit (IC) temperature sensor
- Operating Range: -55°C to +150°C
- Output: Linear analog voltage — 10 mV per °C (e.g., 25°C → 250 mV output)
- Supply Voltage: 4V to 30V DC
- Accuracy: ±0.5°C typical at room temperature
- Interface: Connected to Analog-to-Digital Converter (ADC) pin on microcontroller
- Formula: Temperature (°C) = $V_{out} \text{ (mV)} \div 10$
- Advantage: No calibration required, low cost, easy to use with Arduino ADC

- Limitation: Analog output requires ADC on microcontroller; not suitable for noisy environments
- Applications: Room thermometers, HVAC control, industrial temperature monitoring

🔧 DHT11 — Digital Temperature & Humidity Sensor

- Type: Digital output — measures BOTH temperature and humidity
- Temperature Range: 0°C to 50°C | Accuracy: $\pm 2^\circ\text{C}$
- Humidity Range: 20% to 80% RH | Accuracy: $\pm 5\%$ RH
- Output: Single-wire digital protocol (1-Wire protocol variant)
- Supply Voltage: 3.3V to 5V DC
- Sampling Rate: Once per second (1 Hz) — cannot take readings faster
- Communication: Sends 40-bit data packet (8-bit humidity int + 8-bit humidity dec + 8-bit temp int + 8-bit temp dec + 8-bit checksum)
- Advantage: Dual measurement (temp + humidity), digital output (no ADC needed), cheap (~₹50)
- Limitation: Slower sampling rate, lower accuracy than DHT22
- Applications: Weather stations, home automation, greenhouse monitoring

🔧 DHT22 — High-Accuracy Digital Temperature & Humidity Sensor

- Type: Digital output — improved version of DHT11
- Temperature Range: -40°C to +80°C | Accuracy: $\pm 0.5^\circ\text{C}$
- Humidity Range: 0% to 100% RH | Accuracy: $\pm 2\text{--}5\%$ RH
- Output: Single-wire digital protocol (same as DHT11 but 16-bit resolution)
- Supply Voltage: 3.3V to 6V DC | Sampling Rate: 0.5 Hz (once per 2 seconds)
- Advantage: Much wider range and better accuracy than DHT11
- Cost: Higher than DHT11 (~₹150–200)
- Applications: Industrial monitoring, precision agriculture, server room monitoring

🔧 DS18B20 — Waterproof Digital Temperature Sensor

- Type: Digital, 1-Wire protocol — unique among temperature sensors
- Temperature Range: -55°C to +125°C | Accuracy: $\pm 0.5^\circ\text{C}$
- Key Feature: Each sensor has a unique 64-bit serial number — multiple DS18B20s can share a SINGLE data wire
- Supply Voltage: 3.0V to 5.5V | Parasitic power mode (can power itself from the data line)
- Resolution: Configurable — 9-bit to 12-bit (0.5°C to 0.0625°C resolution)
- Advantage: Waterproof probe version available, multi-drop network on single wire, long cable runs possible
- Applications: Liquid/pipe temperature monitoring, aquariums, pool temperature, soil temperature in precision farming

Thermocouple (Type K) — High-Temperature Sensor

- Type: Analog, passive (generates its own voltage — Seebeck effect)
- Operating Principle: Two dissimilar metals joined at one end; temperature difference between junction and reference generates a small voltage (millivolts)
- Range: Type K (Chromel-Alumel): -200°C to +1372°C — widest range of common thermocouples
- Output: Very small analog voltage (~41 $\mu\text{V}/^\circ\text{C}$) — requires amplifier IC (MAX6675 or MAX31855) for microcontroller interfacing
- Advantage: Extremely wide temperature range, rugged, suitable for harsh environments
- Limitation: Low signal amplitude requires amplification; cold-junction compensation needed
- Applications: Industrial furnaces, engine temperature monitoring, kilns, 3D printer hot end

1.2.1 Temperature Sensor Comparison Table

Sensor	Output Type	Temp Range	Accuracy	Interface	Best For
LM35	Analog	-55 to +150°C	$\pm 0.5^\circ\text{C}$	ADC pin	Simple analog circuits
DHT11	Digital	0 to +50°C	$\pm 2^\circ\text{C}$	1-Wire	Basic projects, cheap
DHT22	Digital	-40 to +80°C	$\pm 0.5^\circ\text{C}$	1-Wire	Better accuracy needed
DS18B20	Digital	-55 to +125°C	$\pm 0.5^\circ\text{C}$	1-Wire (multi-drop)	Waterproof, multiple sensors
Thermocouple K	Analog (μV)	-200 to +1372°C	$\pm 2^\circ\text{C}$	SPI (MAX6675)	Extreme temperatures

1.3 Motion Sensors

Purpose: Detect movement of objects or people in the sensor's detection zone. Critical for security systems, automatic lighting, and gesture recognition.

PIR Sensor (Passive Infrared) — HC-SR501

- Type: Passive (no IR emission) — detects infrared radiation emitted by warm bodies
- Operating Principle: All warm objects emit infrared radiation. The PIR sensor detects changes in IR levels as a warm body (human, animal) moves through its detection field.
- Output: Digital HIGH (3.3V/5V) when motion detected, LOW when no motion

- ▶ Detection Range: 3 to 7 meters (adjustable) | Detection Angle: 120° cone
- ▶ Supply Voltage: 5V to 12V DC
- ▶ Key Adjustments (HC-SR501 has two potentiometers): Sensitivity (range) adjustment and Time delay adjustment (how long output stays HIGH after detection)
- ▶ Modes: Non-retriggerable (output goes LOW after fixed time) vs Retriggerable (resets timer when motion continues) — set by jumper
- ▶ Advantage: Low power, very common, cheap (~₹50-100), doesn't need to 'see' the person (works in dark)
- ▶ Limitation: Cannot distinguish between humans and animals; affected by sudden temperature changes
- ▶ Applications: Security alarms, automatic lighting, people counters, smart home occupancy detection



Ultrasonic Sensor — HC-SR04

- ▶ Type: Active — emits ultrasonic sound pulses and measures echo return time
- ▶ Operating Principle: Triggers a 40 kHz ultrasonic burst (8 pulses) from the Trig pin. The Echo pin goes HIGH for the duration of time the sound takes to travel to the object and return.
- ▶ Distance Formula: $\text{Distance (cm)} = (\text{Echo pulse duration in } \mu\text{s}) \times 0.0343 / 2$
- ▶ Range: 2 cm to 400 cm | Accuracy: ± 3 mm
- ▶ Supply Voltage: 5V DC | Trig Pulse: 10 μs HIGH signal
- ▶ Beam Angle: $\sim 15^\circ$ effective cone — objects must be within this cone to be detected
- ▶ Advantage: Non-contact distance measurement, works in the dark, inexpensive (~₹80)
- ▶ Limitation: Not suitable for soft/absorbent materials (like foam); affected by temperature (sound speed changes with temp)
- ▶ Applications: Obstacle detection in robots, parking sensors, liquid level measurement, proximity detection, object counting



MPU-6050 — 6-Axis IMU (Accelerometer + Gyroscope)

- ▶ Type: Digital MEMS (Micro-Electro-Mechanical Systems) sensor — measures motion in 3D space
- ▶ Accelerometer: Measures linear acceleration along X, Y, Z axes (range: $\pm 2g/\pm 4g/\pm 8g/\pm 16g$ selectable)
- ▶ Gyroscope: Measures angular velocity (rotation rate) along X, Y, Z axes (range: $\pm 250/\pm 500/\pm 1000/\pm 2000$ °/s)
- ▶ Interface: I2C (address 0x68 or 0x69) or SPI
- ▶ Supply Voltage: 3.3V to 5V | Built-in 16-bit ADC for high resolution
- ▶ DMP (Digital Motion Processor): On-chip processor that performs quaternion-based motion processing
- ▶ Advantage: 6-axis measurement in one compact IC, highly integrated, widely supported libraries

- Applications: Drone stabilization/flight control, wearable fitness trackers (step counting), gesture recognition, tilt detection, vehicle tracking

1.4 Proximity Sensors

Purpose: Detect the presence or absence of an object within a certain distance WITHOUT physical contact. Essential for industrial automation, touchless interfaces, and obstacle avoidance.

● IR Proximity Sensor (Infrared)

- Operating Principle: Emits infrared light from an IR LED and uses a photodetector to measure reflected IR light. More reflected light = object is closer.
- Output: Digital (HIGH/LOW threshold comparator — common module) or Analog (raw reflected intensity)
- Detection Range: Typically 2 cm to 30 cm (varies by module) — short range sensor
- Supply Voltage: 3.3V to 5V | Adjustable sensitivity via potentiometer
- Limitation: Surface color matters — black/dark surfaces absorb IR (harder to detect) while white/reflective surfaces are easier
- Limitation: Sunlight interference can cause false readings (sunlight contains IR)
- Applications: Line-following robots (detect black/white lines), obstacle avoidance, touchless switch, object detection on conveyor belts

● Capacitive Proximity Sensor

- Operating Principle: Detects changes in capacitance caused by the presence of any object (conductive or non-conductive). The sensing face acts as one plate of a capacitor.
- Detection Range: 1 mm to 20 mm typically
- Key Advantage over Inductive: Can detect non-metallic objects (plastic, wood, glass, liquid, food, human body)
- Output: Digital (normally open or normally closed) switching output
- Applications: Liquid level detection through container walls, touch sensing (capacitive touchscreens), detecting non-metallic objects in food processing

⚙ Inductive Proximity Sensor

- Operating Principle: Generates an electromagnetic field from a coil. When a metallic object enters the field, eddy currents are induced in the metal, changing the coil's inductance and triggering the output.
- Detection Range: 1 mm to 60 mm (depends on target metal)
- Key Feature: Detects ONLY metallic (ferrous and non-ferrous) objects — completely ignores non-metals
- Target Factor: Ferrous metals (iron, steel) are detected at maximum rated distance; non-ferrous metals (aluminum, copper) at ~30-40% of rated distance

- ▶ **Advantage:** Highly reliable, immune to dust/dirt/moisture, long service life, no mechanical contact
- ▶ **Applications:** CNC machine tool positioning, gear tooth counting, metal object detection on assembly lines, valve position sensing



SHARP GP2Y0A21 — Analog IR Distance Sensor

- ▶ **Type:** Analog, triangulation-based distance measurement
- ▶ **Operating Principle:** Uses triangulation — measures the angle at which reflected IR light returns to calculate distance more accurately than simple intensity-based IR sensors
- ▶ **Range:** 10 cm to 80 cm | **Output:** Analog voltage (inversely proportional to distance — closer = higher voltage)
- ▶ **Supply Voltage:** 4.5V to 5.5V
- ▶ **Advantage:** More reliable than simple IR proximity sensors; provides actual distance data, not just presence/absence
- ▶ **Applications:** Obstacle avoidance in robots, automatic faucets, hand dryers, interactive displays

1.5 Gas Sensors

Purpose: Detect the presence and concentration of specific gases in the environment. Critical for safety monitoring (leak detection), air quality assessment, and industrial process control.

MQ Sensor Series Overview: The MQ (Metal Oxide Semiconductor) sensor series is the most widely used family of gas sensors in IoT projects. They operate by measuring changes in electrical resistance of a metal oxide sensing element when gas molecules adsorb onto its surface.

Sensor	Target Gas(es)	Detection Range	Applications
MQ-2	LPG, Propane, Methane, Smoke, H ₂ , Alcohol	300–10000 ppm	LPG leak alarm, smoke detector, inflammable gas detection
MQ-3	Alcohol (Ethanol)	0.05–10 mg/L	Breathalyzer, vehicle drunk-driving prevention
MQ-4	Methane (Natural Gas), CNG	300–10000 ppm	Natural gas leak detection in homes and factories
MQ-5	LPG, Natural Gas, Coal Gas	300–10000 ppm	Fuel gas leak detector, city gas system
MQ-6	LPG, Butane, Propane	200–10000 ppm	LPG leak alarm in households and restaurants
MQ-7	Carbon Monoxide (CO)	20–2000 ppm	CO poisoning alarm, vehicle exhaust monitoring
MQ-8	Hydrogen Gas (H ₂)	100–10000 ppm	Hydrogen fuel cell monitoring, battery room safety

MQ-9	CO + Flammable Gases (LPG, CH4)	10–1000 ppm (CO)	Dual-purpose safety alarm
MQ-135	Air Quality (NH3, CO2, Alcohol, Benzene, NOx)	10–300 ppm	Air quality monitor, indoor CO2/ammonia detection

1.5.1 How MQ Gas Sensors Work — Detailed Explanation

1. The sensing element is a small ceramic tube coated with a metal oxide material (typically SnO2 — tin dioxide).
2. An internal heater coil heats the sensing element to 200–300°C (optimal operating temperature for gas detection).
3. At this temperature, oxygen from air adsorbs on the SnO2 surface, creating a high-resistance layer.
4. When target gas molecules contact the heated surface, they react with adsorbed oxygen, releasing electrons and REDUCING the resistance of the sensing element.
5. A voltage divider circuit converts this resistance change into a measurable analog voltage output.
6. Higher gas concentration → lower sensor resistance → higher analog voltage output.

MQ Sensor Property	Details
Output Type	Analog (variable voltage, 0V–5V range) AND Digital (threshold comparator output via onboard potentiometer)
Preheat Time	Critical — requires 20–48 hours burn-in (first use) and 2–3 minutes warm-up before each use for accurate readings
Supply Voltage	5V DC (heater + circuit)
Load Resistor	A fixed resistor in series forms a voltage divider with the sensor's variable resistance
Sensitivity Adjustment	Onboard potentiometer adjusts the digital output trigger threshold
Cross-Sensitivity	MQ sensors respond to multiple gases — calibration needed for specificity
Cost	Very affordable — ₹50–150 per module
Limitation	Not suitable for precise ppm measurement without detailed calibration curves

1.5.2 DHT11 vs MQ-2 — Comparison

Property	Details
Full Name	DHT11 = Digital Humidity & Temperature MQ-2 = Metal Oxide Semiconductor Gas Sensor

Measures	DHT11: Temperature + Humidity MQ-2: LPG, Smoke, Methane, CO, Alcohol gases
Output	DHT11: Digital 40-bit serial MQ-2: Analog voltage + Digital threshold
Warm-up	DHT11: ~1 second MQ-2: 2–3 minutes (+ 48 hr initial burn-in)
Power	DHT11: 3.3–5V, ~2.5 mA MQ-2: 5V, ~150 mA (heater draws significant current)
Cost	DHT11: ~₹50 MQ-2: ~₹80–120

2. Actuators in IoT

Actuator — Formal Definition

An actuator is a device that receives an electrical signal from a controller (microcontroller, PLC, or computer) and converts it into a physical action or movement in the real world. While sensors are the 'inputs' of an IoT system (gathering data), actuators are the 'outputs' (making things happen).

Sensors vs Actuators: Sensors convert physical phenomena into electrical signals (input devices). Actuators convert electrical signals into physical action (output devices). Together, they form the perception-action loop of IoT.

2.1 Motors — Types and Operation

Definition: A motor is an electromechanical actuator that converts electrical energy into rotational or linear mechanical motion.

2.1.1 DC Motor (Direct Current Motor)

DC Motor

- ▶ Operating Principle: When current flows through a wire coil (armature) placed in a magnetic field, the Lorentz force creates torque, causing rotation (Fleming's Left-Hand Rule).
- ▶ Speed Control: Speed is proportional to applied voltage — use PWM (Pulse Width Modulation) to control speed digitally
- ▶ Direction Control: Reversing current direction reverses rotation direction
- ▶ Key Component: H-Bridge driver IC (L298N, L293D) required to control DC motors from a microcontroller (microcontroller GPIO cannot supply enough current directly)
- ▶ L298N Driver: Can drive TWO DC motors independently | Input: 5V logic, 6–12V motor supply | Output: Up to 2A per channel
- ▶ PWM Speed Control: Arduino `analogWrite(pin, 0–255)` — 0 = stopped, 255 = full speed
- ▶ Applications: Wheeled robots, conveyor belts, fans, pumps, RC cars, camera pan/tilt systems
- ▶ Limitations: Brushed DC motors have wear (brush/commutator), generates electrical noise, no inherent position feedback

2.1.2 Servo Motor

Servo Motor (SG90, MG996R)

- ▶ Type: Closed-loop DC motor with built-in gearbox, position sensor (potentiometer), and controller circuit

- ▶ Operating Principle: Receives PWM signal — the pulse width determines the angular position (not speed)
- ▶ Control Signal: PWM at 50 Hz (20 ms period). Pulse width: 1 ms → 0°, 1.5 ms → 90°, 2 ms → 180°
- ▶ Angular Range: Standard servo = 0° to 180° | Continuous rotation servo = 360° (modified for robotics)
- ▶ SG90 Specs: Torque = 1.8 kg·cm, Speed = 0.1 sec/60°, Voltage = 4.8–6V, Weight = 9g (micro servo)
- ▶ MG996R Specs: Metal gears, Torque = 9.4–11 kg·cm (much stronger), suitable for heavy-duty robotic arms
- ▶ Arduino Control: Uses Servo.h library — `servo.write(angle)` where angle = 0–180
- ▶ Advantage: Precise position control without additional encoder; easy to control
- ▶ Applications: Robot arm joints, RC aircraft control surfaces, camera pan/tilt, robotic gripper, door locks

2.1.3 Stepper Motor (28BYJ-48 + ULN2003)



Stepper Motor

- ▶ Operating Principle: Rotates in precise discrete steps by energizing stator coils in a specific sequence. Each step = fixed angular increment.
- ▶ 28BYJ-48 Specs: 5V unipolar stepper | Steps per revolution: 2048 steps (with 64:1 gearbox) | Step angle = $5.625^\circ/64 = 0.088^\circ$ effective
- ▶ Driver: ULN2003 driver board (Darlington transistor array) — required to drive the 4 coil phases
- ▶ Control Modes: Full-step (4 steps), Half-step (8 steps for finer resolution), Microstepping
- ▶ Step Sequence (4-step full-step): IN1-IN2-IN3-IN4 → 1000, 0100, 0010, 0001 (repeat)
- ▶ Key Advantage: Open-loop precise positioning — no feedback sensor needed, always knows exact position (counts steps)
- ▶ Limitation: Lower speed and torque compared to servo/DC motors; vibration at low speeds
- ▶ Applications: 3D printer axes (X/Y/Z movement), CNC machines, camera autofocus, precise valve control, document scanner

2.1.4 Motor Comparison

Feature	DC Motor	Servo Motor	Stepper Motor
Control Type	Speed + Direction (PWM)	Angular Position (PWM)	Step-by-step position (digital)
Feedback	Open loop (no position feedback)	Closed loop (built-in pot)	Open loop (counts steps)
Precision	Low (no position control)	High ($\pm 1-2^\circ$)	Very High (fraction of a degree)

Speed	High RPM possible	Limited to ~60–120 RPM	Low to medium RPM
Torque	High at low speed	Medium (depends on model)	Good holding torque
Driver IC	L298N / L293D	Direct PWM from MCU	ULN2003 / A4988
Complexity	Low	Very Low (1 PWM signal)	Medium (sequence control)
Cost	Low	Low to Medium	Medium
Best For	Wheels, fans, pumps	Robot joints, RC control	3D printers, CNC, precision positioning

2.2 Relays

Relay — Definition

A relay is an electrically operated switch that uses a small control signal (from a microcontroller, typically 5V/3.3V, few mA) to switch a completely separate, high-power circuit (up to 250V AC, 10A or more). It provides electrical isolation between the control circuit and the load circuit.

2.2.1 How a Relay Works

7. The microcontroller sends a LOW or HIGH signal to the relay module's input pin.
8. This signal activates a transistor in the relay driver circuit, which allows current to flow through the relay's electromagnet coil.
9. The electromagnetic field generated by the coil attracts a ferromagnetic armature.
10. The moving armature switches the mechanical contacts — connecting or disconnecting the high-power circuit.
11. When the control signal is removed, a spring returns the armature to its original position, and a flyback diode protects the circuit from the voltage spike created when the coil de-energizes.

2.2.2 Relay Contact Types

Contact Type	Description & Use
NO (Normally Open)	Contact is OPEN (circuit broken) when relay coil is NOT energized. Contact CLOSES when relay is activated. Most common — circuit is OFF by default, turns ON when relay triggers.
NC (Normally Closed)	Contact is CLOSED (circuit connected) when relay coil is NOT energized. Contact OPENS when relay activates. Use when you want circuit ON by default, turned OFF by relay.
COM (Common)	The common terminal shared between NO and NC contacts. Always connected to the load's power supply.

2.2.3 5V Relay Module Specifications

Specification	Value / Description
Control Voltage	5V DC (compatible with Arduino, ESP32)
Control Signal	Typically Active LOW — GPIO LOW = relay ON; GPIO HIGH = relay OFF
Coil Current	~70–80 mA (too high for direct GPIO — uses transistor driver on module)
Switching Capacity	10A @ 250V AC or 10A @ 30V DC
Isolation	Complete electrical isolation between control and load circuits
LED Indicator	Most modules have LED showing relay state
Flyback Diode	Built into most modules — protects against inductive spike when coil de-energizes
Applications	Controlling AC appliances (lights, fans, heaters), solenoids, motors from microcontrollers

2.3 Valves (Solenoid Valves)



Solenoid Valve — Definition

A solenoid valve is an electromechanical valve that controls the flow of liquids or gases. It uses an electromagnet (solenoid coil) to move a plunger that opens or closes the valve orifice. In IoT, solenoid valves are used to remotely control fluid flow without physical intervention.

2.3.1 How a Solenoid Valve Works

- At rest (de-energized), a spring holds the plunger in the CLOSED position, blocking fluid flow.
- When current flows through the solenoid coil, it generates a magnetic field.
- The magnetic field pulls the iron plunger upward (or downward), compressing the spring.
- The lifted plunger opens the orifice, allowing fluid/gas to flow through.
- When power is removed, the spring pushes the plunger back, closing the valve.

Type	Description & Applications
Normally Closed (NC)	Valve is CLOSED when de-energized. Opens when current applied. MOST COMMON for fail-safe applications — ensures flow stops if power is lost.
Normally Open (NO)	Valve is OPEN when de-energized. Closes when current applied. Used where flow should continue if power fails.
2-Way Valve	One inlet, one outlet — simply opens or closes flow. Most basic valve.

3-Way Valve	Three ports — can divert flow between two paths. Used in mixing or diverting applications.
Proportional Valve	Controls flow rate proportionally (not just ON/OFF) — requires analog or PWM control signal. Used in precise flow control.

- **IoT Applications:** Smart irrigation systems (watering plants automatically based on soil moisture sensor), water leak shutoff (close main valve when leak sensor detects water), industrial process control, smart aquariums
- **Interface with Arduino:** Solenoid valves require 12V or 24V — controlled via relay module or MOSFET driver from 5V Arduino GPIO

⚡ Key Points to Remember

- Sensors = INPUT (convert physical → electrical) | Actuators = OUTPUT (convert electrical → physical)
- PIR sensor: detects infrared from warm bodies — digital output — HC-SR501
- HC-SR04: ultrasonic distance = (echo duration μs $\times$ 0.0343) / 2 cm
- MQ sensors: need 20–48 hr burn-in + 2–3 min warm-up before each use
- Servo motor: 1ms = 0°, 1.5ms = 90°, 2ms = 180° (PWM pulse width control)
- Stepper motor: open-loop precise control, 2048 steps/revolution (28BYJ-48)
- Relay: NO = off by default, NC = on by default, COM = load connection
- Solenoid valve: electromagnet opens/closes fluid flow — controlled via relay from MCU

3. Introduction to Arduino

Arduino — Formal Definition

Arduino is an open-source electronics platform (hardware + software) based on easy-to-use hardware (microcontroller boards) and software (Arduino IDE). It is designed for both beginners and experts to create interactive electronic projects. The hardware consists of microcontroller-based circuit boards; the software is a cross-platform IDE using a simplified version of C/C++.

3.1 Arduino Uno — The Standard Reference Board

The Arduino Uno is the most widely used Arduino board and the standard for learning. Understanding its components is essential.

3.1.1 Arduino Uno Hardware Specifications

Component	Specification	Notes / Importance
Microcontroller	ATmega328P (8-bit AVR)	The brain — runs your code; manufactured by Microchip
Clock Speed	16 MHz	Executes ~16 million instructions per second
Flash Memory	32 KB (program storage)	Stores your compiled sketch; 0.5 KB used by bootloader
SRAM	2 KB	Runtime memory for variables — very limited, manage carefully
EEPROM	1 KB	Non-volatile storage — survives power off; use EEPROM.h library
Operating Voltage	5V	Logic level; input pins operate at 5V
Input Voltage	7–12V (recommended)	Through DC barrel jack or VIN pin; onboard regulator converts to 5V
Digital I/O Pins	14 pins (D0–D13)	Each can source/sink 40mA max; D0/D1 = UART RX/TX
PWM Pins	6 pins (~3,5,6,9,10,11)	8-bit PWM — use analogWrite(); frequency ~490 Hz or ~980 Hz
Analog Input Pins	6 pins (A0–A5)	10-bit ADC — reads 0–5V as 0–1023 digital value
I2C Pins	A4 (SDA), A5 (SCL)	For I2C communication — shared with analog inputs
SPI Pins	D10 (SS), D11 (MOSI), D12 (MISO), D13 (SCK)	For SPI communication; SCK = clock
UART	D0 (RX), D1 (TX)	Serial communication; also used for USB upload — disconnect when uploading

USB Interface	ATmega16U2 (USB-to-serial)	Allows USB upload and Serial Monitor communication
Reset Button	Physical button	Restarts the sketch from the beginning
Power Pins	5V, 3.3V, GND, VIN	3.3V limited to 50mA; 5V from USB ~500mA

3.2 Arduino Programming Fundamentals

Arduino programs are called 'sketches.' Every sketch has two mandatory functions:

setup() Function	loop() Function
Runs ONCE when Arduino powers on or is reset. Use for: Initializing pins, starting Serial communication, configuring sensors. <pre>void setup() { pinMode(13, OUTPUT); Serial.begin(9600); }</pre>	Runs CONTINUOUSLY in an infinite loop after setup(). Use for: Reading sensors, controlling actuators, all main program logic. <pre>void loop() { digitalWrite(13, HIGH); delay(1000); }</pre>

3.2.1 Core Arduino Functions Reference

Function	Syntax	Description
pinMode()	pinMode(pin, mode)	Sets pin as INPUT, OUTPUT, or INPUT_PULLUP. Must call in setup().
digitalWrite()	digitalWrite(pin, value)	Sets digital output pin HIGH (5V) or LOW (0V).
digitalRead()	int val = digitalRead(pin)	Reads digital input — returns HIGH (1) or LOW (0).
analogRead()	int val = analogRead(pin)	Reads analog voltage on A0–A5 → returns 0 to 1023 (10-bit ADC). 0=0V, 1023=5V
analogWrite()	analogWrite(pin, value)	Outputs PWM on PWM-capable pins. Value 0–255. 0=0% duty, 255=100% duty.
delay()	delay(ms)	Pauses program for specified milliseconds. Blocking — program stops.
millis()	unsigned long t = millis()	Returns milliseconds since Arduino started. Non-blocking timer alternative to delay().
Serial.begin()	Serial.begin(baud)	Starts serial communication at baud rate (e.g., 9600, 115200).
Serial.print()	Serial.print(data)	Prints data to Serial Monitor without newline.
Serial.println()	Serial.println(data)	Prints data to Serial Monitor WITH newline character at end.
Serial.read()	char c = Serial.read()	Reads one byte of incoming serial data.

map()	map(val, fromLow, fromHigh, toLow, toHigh)	Re-maps a number from one range to another. E.g., map(analogVal, 0, 1023, 0, 100)
constrain()	constrain(val, min, max)	Constrains value to range [min, max]. Returns min if below, max if above.

3.3 GPIO Programming

GPIO — General Purpose Input/Output

GPIO (General Purpose Input/Output) pins are programmable digital pins on a microcontroller or SBC that can be configured as either inputs (to read signals from sensors/switches) or outputs (to send signals to LEDs, relays, motors). 'General Purpose' means they are not dedicated to a specific function — the programmer decides how each pin is used.

3.3.1 GPIO as Digital OUTPUT

Configuring a pin as OUTPUT allows the microcontroller to drive external devices:

Concept	Explanation
Source Current (HIGH)	When OUTPUT pin is HIGH (5V), current flows FROM the pin INTO the circuit. Max 40mA per pin on Arduino Uno.
Sink Current (LOW)	When OUTPUT pin is LOW (0V/GND), current flows INTO the pin FROM the circuit. Also max 40mA.
Total GPIO Current	Total current across ALL GPIO pins should not exceed 200mA — use external drivers for high-current loads.
LED Connection	LED on OUTPUT pin: connect 220Ω–1kΩ resistor in series to limit current ($V=IR$, typical LED $V_f=2V$, $I_f=10\text{--}20\text{mA}$)
Relay/Motor Control	Always use driver ICs (L298N, relay module, ULN2003) for high-current loads — never connect motors directly to GPIO
PWM Output	Pulse Width Modulation — rapidly switches pin HIGH/LOW at varying duty cycles to simulate analog voltage. 8-bit = 256 levels.

3.3.2 GPIO as Digital INPUT

Configuring a pin as INPUT allows reading external signals:

Input Mode	Description & When to Use
INPUT (floating)	Pin reads external voltage. When nothing is connected, pin 'floats' — can randomly read HIGH or LOW. AVOID for switches.
INPUT_PULLUP	Internal 10–20kΩ resistor connects pin to VCC (5V). Pin reads HIGH when switch is open; reads LOW when switch is pressed

	(connects to GND). Use with active-LOW switches. RECOMMENDED.
INPUT_PULLDOWN (RP2040)	Internal resistor to GND. Arduino Uno doesn't have this — available on Raspberry Pi Pico, STM32. Pin reads LOW when floating.
Debouncing	Physical switches 'bounce' (make/break contact rapidly for 5–20ms when pressed). Must implement software debounce with <code>millis()</code> or use <code>delay(50ms)</code> to avoid multiple false readings.

3.3.3 GPIO on Raspberry Pi

The Raspberry Pi has 40 GPIO pins with additional special functions:

Pin Type	Voltage	Max Current	Description
GPIO (Digital I/O)	3.3V logic	8–16 mA per pin	Programmable digital I/O. NOTE: 3.3V logic — NOT 5V tolerant! 5V input will damage the Pi.
5V Power Pins	5V	Limited by power supply	Pin 2 and 4 — provide 5V power output (not GPIO)
3.3V Power Pins	3.3V	~50 mA total	Pin 1 and 17 — provide 3.3V power output
GND Pins	0V	—	Multiple GND pins (6, 9, 14, 20, 25, 30, 34, 39)
I2C	3.3V	—	GPIO 2 (SDA), GPIO 3 (SCL) — I2C bus 1
SPI	3.3V	—	GPIO 10 (MOSI), GPIO 9 (MISO), GPIO 11 (SCLK), GPIO 8/7 (CE0/CE1)
UART	3.3V	—	GPIO 14 (TXD), GPIO 15 (RXD) — serial communication
PWM	3.3V	—	GPIO 12, 13, 18, 19 have hardware PWM capability

GPIO Programming on Raspberry Pi — RPi.GPIO Library (Python):

```
import RPi.GPIO as GPIO
import time

# Setup — choose numbering scheme
GPIO.setmode(GPIO.BCM) # BCM = GPIO number | GPIO.BOARD = physical pin number

# Configure pins
GPIO.setup(18, GPIO.OUT) # GPIO 18 as output (LED)
GPIO.setup(17, GPIO.IN, pull_up_down=GPIO.PUD_UP) # Button with pull-up

try:
    while True:
        if GPIO.input(17) == GPIO.LOW: # Button pressed (active low)
```

```

        GPIO.output(18, GPIO.HIGH) # Turn LED ON
    else:
        GPIO.output(18, GPIO.LOW) # Turn LED OFF
    time.sleep(0.05) # 50ms debounce delay
except KeyboardInterrupt:
    GPIO.cleanup() # ALWAYS cleanup — resets all pins to safe state

```

3.4 PWM (Pulse Width Modulation) — In Depth



PWM — Definition

Pulse Width Modulation (PWM) is a technique used to simulate an analog output using a digital signal by varying the proportion of time the signal is HIGH vs LOW in a fixed period. The percentage of time the signal is HIGH is called the Duty Cycle.

PWM Parameter	Explanation
Period (T)	The fixed time for one complete cycle. For Arduino PWM: $T \approx 2\text{ms}$ (490 Hz) or 1ms (980 Hz) depending on pin.
Frequency (f)	Number of PWM cycles per second. $f = 1/T$. Arduino default: ~490 Hz (most pins), ~980 Hz (pins 5 & 6).
Duty Cycle (%)	Percentage of the period the signal is HIGH. Duty Cycle = $(\text{ON time} / \text{Period}) \times 100\%$
analogWrite(pin, 0)	0% duty cycle — signal always LOW — effective output = 0V
analogWrite(pin, 128)	~50% duty cycle — signal half HIGH, half LOW — effective output $\approx 2.5\text{V}$
analogWrite(pin, 255)	100% duty cycle — signal always HIGH — effective output = 5V
LED Brightness	PWM duty cycle directly controls perceived LED brightness (eye sees average)
Motor Speed	Average voltage applied to motor $\propto$ duty cycle $\rightarrow$ controls motor speed smoothly

4. Introduction to Raspberry Pi

Raspberry Pi — Definition

The Raspberry Pi is a series of small, low-cost, credit-card-sized single-board computers (SBC) developed by the Raspberry Pi Foundation (UK) to promote computer science education. Unlike microcontrollers (Arduino), the Raspberry Pi runs a full Linux operating system, making it a complete computer with networking, file system, and the ability to run any software that Linux supports.

4.1 Raspberry Pi Models Overview

Model	Processor	RAM	Connectivity	Key Features
RPi Zero 2 W	Quad-core A53 @ 1 GHz	512 MB	Wi-Fi, BT 4.2, mini HDMI	Ultra-compact, low power — wearables, tiny IoT
RPi 3 Model B+	Quad-core A53 @ 1.4 GHz	1 GB	Wi-Fi ac, BT 4.2, GbE	Most educational RPi — still widely used
RPi 4 Model B	Quad-core A72 @ 1.5 GHz	1/2/4/8 GB	Wi-Fi ac, BT 5.0, GbE	Desktop-class performance, USB 3.0, 4K HDMI
RPi 5	Quad-core A76 @ 2.4 GHz	4/8 GB	Wi-Fi ac, BT 5.0, GbE	2–3× faster than RPi 4, PCIe connector for NVMe
RPi Pico (W)	Dual-core M0+ @ 133 MHz	264 KB SRAM	Pico W: Wi-Fi, BT	Microcontroller (not SBC) — closest to Arduino, MicroPython

4.2 Raspberry Pi vs Arduino — Detailed Comparison

Feature	Raspberry Pi 4	Arduino Uno
Category	Single-Board Computer (SBC)	Microcontroller Development Board
Processor	Quad-core ARM Cortex-A72, 64-bit, 1.5 GHz	8-bit AVR ATmega328P, 16 MHz
RAM	1 GB – 8 GB LPDDR4	2 KB SRAM
Storage	MicroSD card (8 GB+)	32 KB Flash (program only)
OS	Linux (Raspberry Pi OS), Ubuntu, others	No OS — runs one program directly
Programming Languages	Python, C/C++, Java, Node.js, Scratch, any	C/C++ (Arduino IDE)

Multitasking	Yes — full OS multitasking (run multiple programs)	No — single program only
Internet Connectivity	Built-in Wi-Fi + Gigabit Ethernet	Requires add-on shield
GPIO Voltage	3.3V (NOT 5V tolerant!)	5V (also 3.3V on some models)
Startup Time	30–60 seconds (Linux boot)	Instant (<1 second)
Real-Time Control	Not ideal (OS scheduling delays)	Excellent (bare metal, deterministic timing)
Power Consumption	3–7 Watts (RPI 4)	0.05–0.25 Watts
Cost	~₹3,000–6,000	~₹500–2,000
Best For	Gateway, edge AI, computer vision, web server, complex processing	Sensor reading, motor control, real-time control, simple IoT nodes
Shutdown	Must shut down gracefully (shutdown -h now) — SD card corruption if power cut	Can cut power instantly — no OS to corrupt

4.3 Raspberry Pi as IoT Gateway

The Raspberry Pi's most powerful IoT role is as an IoT Gateway — collecting data from multiple local sensors/devices and forwarding to the cloud:

- **Local Data Collection:** Raspberry Pi reads data from Arduinos via serial/USB, from sensors via GPIO/I2C/SPI, or from other devices via Wi-Fi/Bluetooth
- **Edge Processing:** Runs ML models locally (TensorFlow Lite) for inference without cloud dependency — reduces latency and bandwidth
- **Protocol Translation:** Converts local protocols (Zigbee, Bluetooth, Modbus) to cloud protocols (MQTT, HTTP) — acts as a bridge
- **MQTT Broker:** Can run Mosquitto (MQTT broker) locally — all local devices publish to RPi broker, which then forwards to cloud
- **Web Dashboard:** Run Flask/Node.js web server to display sensor data on a local dashboard accessible via browser

5. Sensor-Controller Interfacing

Sensor-Controller Interfacing refers to the process of connecting sensors and actuators to a microcontroller or SBC and reading/writing data through the appropriate communication protocol or GPIO interface. Understanding interfacing protocols is essential for every IoT project.

5.1 Communication Interfaces/Protocols for Sensor Interfacing

Protocol	Type	Wires Required	Speed	Devices	Best For
GPIO Digital	Digital I/O	1 data + GND	Instantaneous	Unlimited (separate pins)	Buttons, LEDs, relays, digital sensors
ADC (Analog)	Analog Input	1 data + GND	Instantaneous	1 per ADC pin	LM35, potentiometers, analog sensors
UART / Serial	Async Serial	2 (TX, RX) + GND	9600–115200 bps	1 device per UART	GPS, Bluetooth HC-05, Serial displays
I2C / TWI	Sync Serial (2-wire)	2 (SDA, SCL) + GND/VCC	100K / 400K / 1M bps	Up to 127 devices on 1 bus	Multiple sensors (MPU6050, OLED, BMP280)
SPI	Sync Serial (4-wire)	4 (MOSI, MISO, SCK, CS) + GND	Up to 10 MHz+	Multiple (1 CS pin per device)	SD cards, TFT displays, MAX6675
1-Wire	Async Single-Wire	1 data (parasite power) + GND	15.4 Kbps	Multiple (unique 64-bit IDs)	DS18B20 temperature sensors
PWM	Analog-equivalent Output	1 PWM + GND	490–50k Hz	1 per PWM pin	Servo motors, LED brightness, motor speed

5.2 I2C Protocol — In Depth

I2C (Inter-Integrated Circuit)

I2C (pronounced 'I-squared-C') is a synchronous, multi-master, multi-slave, serial communication protocol developed by Philips (now NXP). It uses only TWO wires — SDA (Serial Data) and SCL (Serial Clock) — to connect multiple devices on a single bus. Each device has a unique 7-bit address (0–127) allowing up to 127 devices on one bus.

5.2.1 I2C Communication Details

I2C Feature	Details
-------------	---------

SDA (Serial Data)	Bidirectional data line — carries data bits between master and slave
SCL (Serial Clock)	Unidirectional clock line — always driven by the master to synchronize data
Pull-up Resistors	Both SDA and SCL REQUIRE pull-up resistors (typically 4.7kΩ) to VCC. Without them, the bus won't work. Many sensor modules include these.
Start Condition	SDA goes LOW while SCL is HIGH — signals beginning of transaction
Stop Condition	SDA goes HIGH while SCL is HIGH — signals end of transaction
ACK/NACK	After each byte, receiver pulls SDA LOW (ACK = acknowledge) or leaves HIGH (NACK = not acknowledge/error)
Address Frame	First 7 bits = device address, last bit = R/W (0 = write, 1 = read)
Speed Modes	Standard: 100 kbps Fast: 400 kbps Fast-Plus: 1 Mbps High-Speed: 3.4 Mbps
Arduino I2C Pins	Uno: A4 (SDA), A5 (SCL) Mega: 20 (SDA), 21 (SCL) Use Wire.h library
Raspberry Pi I2C	GPIO 2 (SDA), GPIO 3 (SCL) — Bus 1. Enable in raspi-config. Use smbus2 library in Python.

5.2.2 Common I2C Sensor Addresses

Sensor / Module	I2C Address	Function
MPU-6050	0x68 (AD0=LOW) or 0x69 (AD0=HIGH)	6-axis IMU (Accelerometer + Gyroscope)
BMP280 / BME280	0x76 or 0x77	Temperature + Pressure (+ Humidity for BME280)
SSD1306 OLED	0x3C or 0x3D	128×64 OLED display
PCF8574	0x20–0x27 (configurable)	8-bit I/O expander (add 8 more GPIO pins)
DS3231 RTC	0x68	Real-Time Clock (note: same address as MPU-6050!)
ADS1115	0x48–0x4B	16-bit 4-channel ADC expander for analog sensors

5.3 Practical Sensor Interfacing Examples

5.3.1 DHT11 Temperature & Humidity Sensor with Arduino

Connection	Details
DHT11 Pin 1 (VCC)	→ Arduino 5V
DHT11 Pin 2 (DATA)	→ Arduino Digital Pin D2 (+ 10kΩ pull-up resistor to 5V)
DHT11 Pin 4 (GND)	→ Arduino GND
Library Required	DHT sensor library by Adafruit (install via Library Manager in IDE)

```
#include <DHT.h>
#define DHTPIN 2      // Data pin connected to D2
#define DHTTYPE DHT11 // Sensor type
DHT dht(DHTPIN, DHTTYPE); // Create DHT object

void setup() {
  Serial.begin(9600);
  dht.begin();        // Initialize sensor
}

void loop() {
  delay(2000);          // DHT11 needs 2s between readings
  float temp = dht.readTemperature(); // Read °C
  float hum  = dht.readHumidity();    // Read %RH
  if (isnan(temp) || isnan(hum)) {    // Check for read error
    Serial.println('DHT read failed!'); return;
  }
  Serial.print('Temp: '); Serial.print(temp); Serial.print(' C ');
  Serial.print('Humidity: '); Serial.print(hum); Serial.println(' %');
}
```

5.3.2 HC-SR04 Ultrasonic Sensor with Arduino

Connection	Details
VCC → 5V	HC-SR04 TRIG → D9 ECHO → D10 GND → GND
Trigger	Send 10μs HIGH pulse to TRIG to initiate measurement
Echo	ECHO pin stays HIGH for duration proportional to distance
Formula	Distance (cm) = pulseIn(ECHO, HIGH) × 0.0343 / 2

```
#define TRIG_PIN 9
#define ECHO_PIN 10
void setup() {
  pinMode(TRIG_PIN, OUTPUT);
  pinMode(ECHO_PIN, INPUT);
  Serial.begin(9600);
}
```



```

}

void loop() {
  // Send 10µs trigger pulse
  digitalWrite(TRIG_PIN, LOW); delayMicroseconds(2);
  digitalWrite(TRIG_PIN, HIGH); delayMicroseconds(10);
  digitalWrite(TRIG_PIN, LOW);

  // Measure echo duration and calculate distance
  long duration = pulseIn(ECHO_PIN, HIGH); // µs
  float distance = duration * 0.0343 / 2.0; // cm
  Serial.print('Distance: '); Serial.print(distance); Serial.println('
cm');
  delay(500); // Wait 500ms between measurements
}

```

5.3.3 PIR Motion Sensor with Arduino — Alarm System

Connection	Details
HC-SR501 Wiring	VCC → 5V OUT → D7 GND → GND
Buzzer Wiring	Buzzer (+) → D8 (through 100Ω resistor) Buzzer (-) → GND
PIR Setting	Set time-delay potentiometer to minimum; set sensitivity as needed
Logic	PIR OUT = HIGH when motion detected → turn buzzer ON LOW = no motion → buzzer OFF

```

#define PIR_PIN 7
#define BUZZER_PIN 8

void setup() {
  pinMode(PIR_PIN, INPUT);
  pinMode(BUZZER_PIN, OUTPUT);
  Serial.begin(9600);
  delay(30000); // PIR sensor needs 30s warmup on startup
}

void loop() {
  int motion = digitalRead(PIR_PIN);
  if (motion == HIGH) { // Motion detected!
    digitalWrite(BUZZER_PIN, HIGH); // Sound alarm
    Serial.println('MOTION DETECTED! Alarm ON');
  } else {
    digitalWrite(BUZZER_PIN, LOW); // Silence alarm
    Serial.println('No motion — area clear');
  }
  delay(100);
}

```

6. Quick Revision Summary & Important Exam Questions

6.1 Quick Revision Table — All Unit II Concepts

Topic	Key Points to Remember
Sensor Definition	Detects physical property → converts to electrical signal. Input device of IoT.
Actuator Definition	Receives electrical signal → converts to physical action. Output device of IoT.
LM35	Analog temp sensor, 10mV/°C output, -55 to +150°C, needs ADC pin
DHT11	Digital temp+humidity, 0–50°C ±2°C, 20–80% RH ±5%, 40-bit protocol, once per second
DHT22	Like DHT11 but better: -40 to +80°C ±0.5°C, 0–100% RH, every 2 seconds
DS18B20	Waterproof, 1-Wire, multi-drop (many sensors on 1 pin), -55 to +125°C
PIR Sensor	Detects infrared from warm bodies. Digital output. HC-SR501. 3–7m range, 120° angle.
HC-SR04	Ultrasonic distance. Trig = 10µs pulse. Distance = echo_µs × 0.0343 / 2 cm. 2–400cm.
MPU-6050	6-axis IMU = 3-axis accelerometer + 3-axis gyroscope. I2C interface. 0x68 address.
MQ Gas Sensors	Metal oxide semiconductor. Needs 48hr burn-in + 3min warm-up. MQ-2=LPG/smoke, MQ-7=CO, MQ-135=air quality
DC Motor	Speed via PWM (analogWrite 0–255), direction via H-bridge (L298N). Cannot connect directly to GPIO.
Servo Motor	Position via PWM. 1ms=0°, 1.5ms=90°, 2ms=180°. 50Hz PWM. Use Servo.h library.
Stepper Motor	Precise discrete steps. 28BYJ-48 = 2048 steps/rev. Driver = ULN2003. Open-loop control.
Relay	NO = off by default, NC = on by default, COM = load power. Active LOW in most modules.
Solenoid Valve	Electromagnet moves plunger to open/close fluid flow. 12V/24V. Controlled via relay.
Arduino Uno	ATmega328P, 16MHz, 2KB RAM, 32KB Flash, 14 digital I/O, 6 ADC, 6 PWM pins, 5V logic
setup() vs loop()	setup() runs once on reset. loop() runs forever. Both are mandatory in every Arduino sketch.
analogRead()	Reads 0–5V on A0–A5 → returns 0–1023 (10-bit ADC). Voltage = reading × 5/1023
PWM	Duty cycle controls average voltage. analogWrite(pin, 0–255). 0%=0V, 100%=5V.

GPIO — RPi	3.3V logic (NOT 5V tolerant!). BCM vs BOARD numbering. Always <code>GPIO.cleanup()</code> at end.
I2C Protocol	SDA + SCL, 2 wires, up to 127 devices, needs pull-up resistors (4.7kΩ), <code>Wire.h</code> library
SPI Protocol	MOSI, MISO, SCK, CS — 4 wires, fast, one CS pin per device, full-duplex
1-Wire Protocol	Single data wire, unique 64-bit device IDs, DS18B20 uses it, parasitic power option
HC-SR04 Code	<code>digitalWrite(TRIG, HIGH)</code> for 10μs → <code>pulseIn(ECHO, HIGH)</code> → $\times 0.0343 / 2$

6.2 Important Exam Questions with Answer Outlines

2-Mark Questions

Q1. Define sensor and actuator. Give one example of each.

Answer: Sensor: device that detects physical property → electrical signal. Example: DHT11 (temperature). Actuator: receives electrical signal → physical action. Example: Servo motor (rotation).

Q2. What is the output of an LM35 sensor at 30°C?

Answer: LM35 outputs 10mV per °C. At 30°C: Output = $30 \times 10\text{mV} = 300\text{mV} = 0.3\text{V}$.

Q3. What is the formula to calculate distance using HC-SR04?

Answer: Distance (cm) = (Echo pulse duration in microseconds $\times 0.0343$) / 2. The /2 accounts for the sound traveling to the object AND back.

Q4. Differentiate between `setup()` and `loop()` in Arduino.

Answer: `setup()`: Runs once when Arduino powers on or resets. Used for initialization (`pinMode`, `Serial.begin`). `loop()`: Runs continuously in infinite loop after `setup()`. Contains main program logic.

Q5. What is GPIO? What is the voltage level of GPIO pins on Raspberry Pi?

Answer: GPIO = General Purpose Input/Output — programmable digital pins that can be configured as inputs or outputs. Raspberry Pi GPIO = 3.3V logic (NOT 5V tolerant).

Q6. Differentiate between NO and NC contacts in a relay.

Answer: NO (Normally Open): Circuit is OPEN (broken) when relay is not energized; closes when activated. NC (Normally Closed): Circuit is CLOSED (connected) when relay is not energized; opens when activated.

4-Mark / 5-Mark Questions — Answer Outlines

Q1. Explain any four types of sensors used in IoT with their working principles and applications.

Answer outline — cover these points:

1. Temperature Sensor (DHT11/LM35): DHT11 = digital, measures temp+humidity via 40-bit protocol; LM35 = analog, 10mV/°C. Applications: HVAC, weather stations.
2. Motion Sensor (PIR HC-SR501): Passive, detects IR from warm bodies. Digital output HIGH when motion detected. Applications: security alarms, auto lighting.
3. Ultrasonic Sensor (HC-SR04): Emits 40kHz pulses, measures echo time. Distance = $\text{echo_}\mu\text{s} \times 0.0343 / 2$. Applications: robotics obstacle detection.
4. Gas Sensor (MQ series): Metal oxide resistance changes with gas concentration. MQ-2=LPG/smoke, MQ-7=CO. Applications: gas leak detection, air quality monitoring.

Q2. Explain DC motor, servo motor, and stepper motor with their control mechanisms.

Answer outline — cover these points:

1. DC Motor: Speed controlled by PWM (analogWrite 0–255), direction by H-bridge (L298N). Open-loop, high speed. Applications: wheels, fans, pumps.
2. Servo Motor: Position controlled by PWM pulse width (1ms=0°, 1.5ms=90°, 2ms=180°). 50Hz frequency. Closed-loop. Applications: robot joints, RC control.
3. Stepper Motor: Discrete steps by energizing coil sequence. 28BYJ-48 = 2048 steps/rev, ULN2003 driver. Open-loop precise positioning. Applications: 3D printers, CNC.
4. Comparison table: control type, feedback, precision, speed, driver IC, best use case.

Q3. Write an Arduino program to read temperature from DHT11 and display on Serial Monitor.

Answer outline — cover these points:

1. Include DHT.h library, define DHTPIN=2, DHTTYPE=DHT11
2. Create DHT object, call dht.begin() in setup()
3. In loop(): read temp with dht.readTemperature(), humidity with dht.readHumidity()
4. Check isnan() for read errors, print values with Serial.println()
5. Add delay(2000) — DHT11 needs 2 seconds between readings
6. Connection: VCC→5V, DATA→D2 + 10kΩ pullup to 5V, GND→GND

Q4. Explain GPIO programming on Arduino and Raspberry Pi with examples.

Answer outline — cover these points:

1. GPIO: General Purpose I/O pins — configurable as input or output by programmer
2. Arduino: `pinMode(pin, OUTPUT/INPUT/INPUT_PULLUP)`, `digitalWrite(HIGH/LOW)`, `digitalRead()`
3. Arduino voltage: 5V logic. PWM on pins 3,5,6,9,10,11 via `analogWrite(0–255)`
4. Raspberry Pi: `RPi.GPIO` library, `GPIO.setmode(BCM)`, `GPIO.setup(pin, IN/OUT)`, `GPIO.output()`, `GPIO.input()`
5. RPi voltage: 3.3V (NOT 5V tolerant!). Always call `GPIO.cleanup()` at end of program.
6. Key difference: Arduino = microcontroller (no OS, real-time), RPi = SBC (Linux OS, more powerful)

⚡ Key Points to Remember

- Sensor = Input device (physical → electrical) | Actuator = Output device (electrical → physical)
- LM35: 10mV/°C output | DHT11: 0–50°C | DHT22: -40 to +80°C (better accuracy) | DS18B20: waterproof, 1-Wire
- HC-SR04 distance formula: $\text{duration}_{\mu\text{s}} \times 0.0343 / 2 = \text{cm}$ — must memorize for exams
- MQ sensors: need 48hr INITIAL burn-in + 2–3 min warm-up before each use
- Servo control: 1ms=0°, 1.5ms=90°, 2ms=180° — PWM at 50Hz
- Arduino Uno: ATmega328P, 16MHz, 2KB RAM, 14 digital + 6 analog pins, 6 PWM pins
- Raspberry Pi GPIO = 3.3V — NEVER connect 5V signals directly to RPi GPIO — will damage the board!
- I2C: 2 wires (SDA + SCL), needs 4.7kΩ pull-ups, up to 127 devices, `Wire.h` library on Arduino

